

Justification for ranges as the output of parallel range algorithms

Document #: P3490R0
Date: 2024-11-14
Project: Programming Language C++
Audience: SG1, SG9
Reply-to: Alexey Kukanov
<alexey.kukanov@intel.com>
Ruslan Arutyunyan
<ruslan.arutyunyan@intel.com>

Contents

- [1 Introduction](#)
- [2 Recap and motivation](#)
- [3 Addressing the concerns](#)
 - [3.1 Mismatch of parallel and serial range algorithms](#)
 - [3.2 Semantic ambiguity for certain algorithms](#)
 - [3.3 Impact on potential future improvements](#)
 - [3.4 What if not accepted for C++26](#)
- [4 Proposed API](#)
- [5 Implementation experience](#)
- [6 Polls](#)
 - [6.1 Joint SG1 + SG9, St. Louis, 2024](#)
- [7 References](#)

§ Abstract

This paper elaborates on the question of using ranges as the output for parallel range algorithms that we proposed in [P3179R2], addressing the raised concerns. The paper neither proposes nor discusses in detail using ranges as the output for serial range algorithms.

§ 1 Introduction

In [P3179R2] we proposed to add overloads taking an execution policy to the functions from [algorithms] defined in the `std::ranges` namespace. We refer to these overloads as *parallel range algorithms*. The proposed design in particular suggested that:

- Parallel range algorithms take a range, not an iterator, as the output for the overloads with ranges, and additionally take an output sentinel for the overloads with iterators and sentinels.
- Parallel range algorithms require `random_access_{iterator, range}`.

- At least one of the input sequences as well as the output sequence must be bounded.

The last two of these design decisions were [approved](#) at the joint SG1 and SG9 meeting in St. Louis. However, the idea of using ranges for the output sequences of parallel range algorithms did not gain sufficient support. The following major concerns were expressed during the meeting as well as in personal communication:

- That would introduce a mismatch between serial and parallel range algorithms so that switching between those will always require more code changes than just adding an execution policy.
- There might be a perception of semantic ambiguity when a whole container is used as the output for certain algorithms, such as in `std::ranges::copy(policy, vec1, vec2)`.
- That could complicate improvements on the output of serial range algorithms, which are considered for the future but will not be ready for C++26.

Consequently, in the [\[P3179R3\]](#) revision we switched back to a single iterator for representing an output sequence. However, with this paper we would like to provide new information and discuss the raised concerns one more time, seeking an opportunity to go forward with the original idea, which we believe will be a notable improvement.

§ 2 Recap and motivation

As a recap, we would like to propose parallel algorithms taking a range as the output for the overloads that take ranges for input. Similarly, we propose adding a sentinel for the output where the input is passed as “iterator and sentinel”. See the [Proposed API](#) section for the examples.

We also propose output ranges to have boundaries set independently of the input ranges. An algorithm should stop as soon as the end is reached for the shortest range. The main motivation is to follow established practices of secure coding, which recommend or even require to always specify the size of the output in order to prevent out-of-range data modifications. We think this will not impose any practical limitation on which ranges can be used for the output of a parallel algorithm, as we could not find or invent an example of a random-access writable range which would also be unbounded.

This *range-as-the-output* approach should not be confused with the `output_range` concept already used with a few serial range algorithms. `output_range` would not be enough for parallel range algorithms, which require random access ranges for the output parameter, same as for the input one.

The benefits of this approach, comparing to taking a single iterator for the output, are:

- It creates a safer API where the modified data sequences have known bounds. Specifically, the `sized_range` and `sized_sentinel_for` concepts will be applied to the output sequences in the same way as [\[P3179R3\]](#) applies those to the input sequences.
- Not for all algorithms the output size is defined by the input size. An example is `copy_if` (and similar algorithms with *filtering* semantics), where the output sequence might be shorter than the input one. Knowing the expected size of the output may open opportunities for more efficient implementations.
- Passing a range for the output makes code a bit simpler in the cases typical for parallel execution.

It is worth noting that to various degrees these reasons are also applicable to serial range algorithms.

We think that in practice parallel algorithms mainly write the output data into a container or storage with preallocated space, for efficiency reasons. So, typically parallel algorithms receive `std::begin(v)` or `v.begin()` or `v.data()` for output, where `v` is an instance of `std::vector` or `std::array`. Allowing `v` to be passed directly for output in the same way as for input results in a slightly simpler code. Here is an example compared to the approach in [\[P3179R3\]](#):

P3179R3	This paper
<pre>void normalize_parallel(random_access_range auto&& v) { auto mx = max(execution::par, v); transform(execution::par, v, views::repeat(mx), ranges::begin(v), divides); }</pre>	<pre>void normalize_parallel(random_access_range auto&& v) { auto mx = max(execution::par, v); transform(execution::par, v, views::repeat(mx), v, divides); }</pre>

In addition, classes such as `std::back_insert_iterator` or `std::ostream_iterator`, which have no meaningful range interpretation, already cannot be used with C++17 parallel algorithms that require at least forward iterators. Code using these types will, in any case, require modifications to migrate to parallel algorithms.

All in all, we think for parallel algorithms taking ranges and sentinels for output makes more sense than only taking an iterator.

Of the 89 parallel range algorithms proposed in [P3179R3] (as counted by names, not overloads), the changes we discuss here will affect only the following 17 algorithms: `copy`, `copy_if`, `move`, `transform`, `replace_copy`, `replace_copy_if`, `remove_copy`, `remove_copy_if`, `unique_copy`, `reverse_copy`, `rotate_copy`, `partition_copy`, `merge`, `set_union`, `set_intersection`, `set_difference`, `set_symmetric_difference`.

§ 3 Addressing the concerns

§ 3.1 Mismatch of parallel and serial range algorithms

The first concern we have heard about this approach is the mismatch between serial and parallel variations. That is, if serial range algorithms only take iterators for output and parallel range algorithms only take ranges, switching between those will always require code changes. That can be resolved by:

- A. adding *range-as-the-output* to serial range algorithms,
- B. adding *iterator-as-the-output* to parallel range algorithms

or both.

The option A would give some of the described benefits to serial range algorithms as well; one could argue that it would be a useful addition on its own. However it is obviously too late to pursue this option for C++26.

The option B does not seem to have benefits besides the aligned semantics, while it has the downside of some algorithm variations not being strengthened with the requirement for the output sequence to be bounded. Nevertheless, given that the option A is not considered for C++26, we prefer the option B, i.e. supporting both ranges and iterators as the output of parallel range algorithms, to the status quo of [P3179R3] that supports only iterators.

For the “iterator and sentinel” overloads we prefer to always require a sentinel for output, despite the mismatch with the corresponding serial overloads. We expect those overloads to be rarely used in general, and in many cases the C++17 parallel algorithms can be used instead. Adding a sentinel for output, on the other hand, preserves the existing approach of expressing a range as the “iterator and sentinel” pair in algorithm descriptions, as well as allows to specify how the end of the output sequence is computed for the option B.

§ 3.2 Semantic ambiguity for certain algorithms

The potential semantic ambiguity can be illustrated by the following code:

```
std::vector<int> vec1;
std::vector<int> vec2;

// Some initialization

std::ranges::copy(policy, vec1, vec2);
```

for which there might be uncertainty what the behavior is when `vec2.size() != vec1.size()`. Some might expect that `std::ranges::copy` resizes `vec2` and makes it an exact copy of `vec1`.

However, the standard already has a precedent in serial variations of `std::ranges::uninitialized_copy` and `std::ranges::uninitialized_move`, which have the range-as-the-output semantics exactly as we propose:

- They use ranges (or sentinels) for both input and output sequences.
- They don't resize the output sequence.
- They stop as soon as any of the sequences reaches its end, copying/moving elements to the minimum of two sizes.

Giving another semantics to `std::ranges::copy` etc. would be inconsistent with these already existing algorithms. Moreover, it would be generally inconsistent with the current *element-wise* semantics of both iterator-based and range algorithms. For example, when `vec2.size()` is greater than `vec1.size()`, `std::ranges::copy(vec1, vec2.begin())` does not modify the elements of `vec2` beyond the size of `vec1`, and so should not `std::ranges::copy(policy, vec1, vec2)`.

Alternatively, potentially ambiguous names can be modified with some prefix, such as `partial_`. It would follow `std::ranges::partial_sort_copy`, another range-as-the-output algorithm that already uses the semantics we propose. That is, we could keep `copy` with iterator-as-the-output and introduce `partial_copy` with range-as-the-output semantics, and similarly for other algorithms. We do not recommend this however, as there is no serial algorithms with such names, and also because it would create very sophisticated names: `partial_remove_copy_if`, really?

Speaking of precedents, it is worth noting that there are more existing range-as-the-output algorithms - `fill`, `generate`, and `iota`. Their specifics is absence of input sequences, so the output sequence needs a boundary. However, extending this principle from algorithms with zero input sequences to those with one or more seems appropriate.

§ 3.3 Impact on potential future improvements

Finally, let's discuss if the proposal could interfere somehow with the anticipated improvements in how the serial range algorithms operate with output data.

First and foremost, the range-as-the-output approach will for now only apply to the new function overloads which first parameter is an execution policy. No modifications to the existing range algorithms are proposed, so there is no direct impact that would limit possible design decisions in the future.

But even though the functions for serial and parallel execution have independent constraints, there is more to consider. If/when the support for ranges as the output of serial algorithms is added later, will it be “backward compatible” with what this paper proposes? In other words, will the transition from a parallel algorithm back to the serial counterpart be as simple as removal of the execution policy?

```
void normalize_serial(range auto&& v) {
    auto mx = max(v);
    transform(v, views::repeat(mx), v, divides);
}
```

The answer, we believe, is “Yes”. The reasoning is that parallel range algorithms impose stricter requirements on their parameters, narrowing down a possible set of arguments. If a range is accepted by a parallel range

algorithm, it is certainly accepted by a serial range algorithm, whether for input or for output. Putting it differently, it will be very strange if serial algorithms somehow exclude random access sized ranges, a subset of the much broader set of writable ranges that could be used as the output.

Another question is whether other anticipated improvements on the output of serial range algorithms might be applicable for the parallel ones, and so should be considered in our design. We have found two WG21 papers that speak about modifications of the output for range algorithms. [P2550R0] proposes that, citing,

- All output algorithms now require `weak_output_iterator`
 - In most cases, that’s re-specifying `weakly_incrementable` and `indirectly_writable` (no requirements change, just better name)
 - In some cases, that’s weakening the requirement on those algorithms that require `output_iterator` (all currently valid code is still valid)

If that paper is accepted, similar requirement modifications might be made for parallel range algorithms, both taking iterators and ranges as the output, and likewise it will not introduce breaking changes for what we propose in [P3179R3] and in this paper.

[P2760R1] discusses shortcomings of *output-only* iterators, such as `std::back_inserter<C>`. While there is no formal proposal yet, the design outlined there uses two new customization points, `std::ranges::put` and `std::ranges::put_range`, which would make implementations of classes like `std::back_inserter` both simpler and more efficient. Types that customize these CPOs would not represent a range, not even an iterator; however, the CPOs themselves would work with output iterators and output ranges. We think that:

- Output-only iterators cannot be used with parallel algorithms, so the use cases that [P2760R1] targets, like `std::back_inserter`, are already not applicable and filtered out.
- Serial range algorithms implemented with these new CPOs should not introduce breaking changes, and therefore could be used as serial blocks in the implementation of parallel range algorithms.
- Additional types that the new CPOs would enable for the output of range algorithms can in principle be accepted by parallel range algorithms if the semantic requirements imposed by execution policies (such as thread safety) are met. It might be discussed and formalized as necessary when these CPOs and possible new concepts based on them will be designed.

To summarize, we do not see how our proposal could create any issues for future improvements related to the output of serial range algorithms, and vice versa.

§ 3.4 What if not accepted for C++26

Since we propose to have both an overload with iterator-as-the-output and an overload with range-as-the-output for each relevant algorithm, it appears that overloads with range-as-the-output could be added later. This is true, though there is a caveat.

If we don’t add the range-as-the-output overload in C++26 that likely means that we don’t add the output sentinel to the “iterator and sentinel” counterpart as well. Given the C++ committee’s concerns about the growing number of algorithm overloads, we are afraid that adding another one with the output sentinel later will be opposed.

That will have the following consequences:

- For the “iterator and sentinel” overloads we will not have a bounded output variation. Having only unbounded one is bad for parallel code, in our opinion.
- There will be no “iterator and sentinel” output pair to fully express the range overload with.

Please look at the “iterator and sentinel” overload in the [Proposed API](#) to see what API we likely lose in the standard if the proposal is not accepted.

§ 4 Proposed API

Below is an example of modifications to be made to the wording proposed in [P3179R3]. The example adjusts the binary transform algorithm to use an output sentinel and adds a new overload taking an output range. Underlines are used to highlight the key differences introduced by this paper. Similar modifications will be made to all the mentioned 17 algorithms if the paper is supported.

Alternatively, we can use an exposition-only *range-or-iterator* concept that combines the requirements for both a range and an iterator by logical disjunction, as its name suggests. We did not explore which way makes more sense; at glance, there seems to be little practical difference for library implementers.

```
template<typename ExecutionPolicy,
        random_access_iterator I1, sentinel_for<I1> S1,
        random_access_iterator I2, sentinel_for<I2> S2,
        random_access_iterator 0, sized_sentinel_for<0> S0,
        copy_constructible F,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_writable<0,
        indirect_result_t<F&, projected<I1, Proj1>, projected<I2, Proj2>>>
        && (sized_sentinel_for<S1, I1> || sized_sentinel_for<S2, I2>)
constexpr binary_transform_result<I1, I2, 0>
transform(ExecutionPolicy&& policy, I1 first1, S1 last1, I2 first2, S2 last2, 0
        result,
        S0 result last, F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});

template<typename ExecutionPolicy,
        ranges::random_access_range R1,
        ranges::random_access_range R2,
        random_access_iterator 0,
        copy_constructible F,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_writable<0,
        indirect_result_t<F&,
        projected<ranges::iterator_t<R1>, Proj1>,
        projected<ranges::iterator_t<R2>, Proj2>>>
        && (sized_range<R1> || sized_range<R2>)
constexpr binary_transform_result<ranges::borrowed_iterator_t<R1>,
        ranges::borrowed_iterator_t<R2>,
        0>
transform(ExecutionPolicy&& policy, R1&& r1, R2&& r2, 0 result, F binary_op,
        Proj1 proj1 = {}, Proj2 proj2 = {});

template<typename ExecutionPolicy,
        ranges::random_access_range R1,
        ranges::random_access_range R2,
        ranges::random_access_range OutR,
        copy_constructible F,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_writable<ranges::iterator_t<OutR>,
        indirect_result_t<F&,
        projected<ranges::iterator_t<R1>, Proj1>,
        projected<ranges::iterator_t<R2>, Proj2>>>
        && (sized_range<R1> || sized_range<R2>) && sized_range<OutR>
constexpr binary_transform_result<ranges::borrowed_iterator_t<R1>,
        ranges::borrowed_iterator_t<R2>,
        ranges::borrowed_iterator_t<OutR>>
transform(ExecutionPolicy&& policy, R1&& r1, R2&& r2, OutR&& result, F binary_op,
        Proj1 proj1 = {}, Proj2 proj2 = {});
```

§ 5 Implementation experience

The oneAPI DPC++ Library (oneDPL) [developer guide](#) covers parallel range algorithms we've implemented so far. The oneAPI specification provides [formal signatures](#) of these algorithms. The implementation supports execution policies for CPUs (semantically aligned with the C++ standard) and for SYCL devices, and it works with a subset of the standard C++ views.

We use the range-as-the-output approach where applicable: in `transform`, `copy`, `copy_if`, and `merge`. We don't have iterator-as-the-output overloads for these algorithms, as [the motivation for it](#) applies much less to oneDPL. However, we don't foresee any issues with adding such overloads later, if necessary.

§ 6 Polls

§ 6.1 Joint SG1 + SG9, St. Louis, 2024

Poll: Continue work on P3179R2 for IS'26 with these notes:

1. RandomAccess for inputs and outputs
2. Iterators for outputs
3. We believe the overloads are worth it

SF	F	N	A	SA
7	4	2	1	0

§ 7 References

[P2550R0] Barry Revzin. 2022-02-17. `ranges::copy` should say `output_iterator` somewhere.

<https://wg21.link/p2550r0>

[P2760R1] Barry Revzin. 2023-12-15. A Plan for C++26 Ranges.

<https://wg21.link/p2760r1>

[P3179R2] Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. 2024-06-25. C++ parallel range algorithms.

<https://wg21.link/p3179r2>

[P3179R3] Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. 2024-10-16. C++ parallel range algorithms.

<https://wg21.link/p3179r3>